GitHub Copilot as a Coding-Agent Harness

Agentic development in VS Code, the Copilot CLI, and GitHub.com

Xebia

Welcome

What This Session Is

- A hands-on tour of GitHub Copilot as a **coding-agent harness**
- One agent loop, one set of tools and context files, one approval model
- Three places it runs: VS Code, the terminal, and GitHub.com
- You'll use all three on a real .NET or Java codebase
- We finish by comparing Copilot with OpenCode and Kiro

Agenda

Time	Block
0:00	Welcome and your practice repo
0:10	Module 1 — The harness and chat in VS Code
1:00	Module 2 — Agent mode, instructions, agents, MCP
1:50	Module 3 — The cloud agent and code review
2:20	Break
2:30	Module 3 activity, part 2
2:50	Module 4 — The Copilot CLI
3:40	Conclusion and questions

How Each Module Runs

- 25 minutes of live demo on the **library** app
- 25 minutes of hands-on work on the **orders** app, in your own repo
- Each activity links its module's slides, for review whenever you like
- You won't finish every task, and that's by design
- Each activity has a `solution/` folder for afterward

Two Apps, Two Languages

App	What it does	Who uses it
Library	Books, loans, due dates, late fees	The instructor's demos
Orders	Catalog, validation, discounts, tax	Your activities

- Each app exists in **.NET 10** and **Java 21**
- Pick the language you work in every day

Your Environment

- A VM with VS Code, Git, the Copilot CLI, the .NET SDK, and a JDK
- The GitHub account you were given, with a Copilot seat
- Check it now (clone the courseware first if it isn't there):

```
git clone https://github.com/coding-agent-harness-class/courseware $HOME\copilot-course
cd $HOME\copilot-course\courseware\setup
powershell -ExecutionPolicy Bypass -File .\setup.ps1 -VerifyOnly
```

- Then sign in: Copilot in VS Code, `copilot` then `/login`, and `gh auth login`

Make Your Practice Repo (1)

- Enterprise accounts can't fork from outside the enterprise, so you **copy** the app
- Your copy is a **private** repo in your own account
- Its Actions tab stays empty: personal repos of enterprise accounts don't run Actions
- Step by step: `courseware/setup/student-quick-start.md`
- Sign in `gh` first, with the `workflow` scope so you can push workflows:

```
gh auth login --web --git-protocol https --scopes workflow
gh auth setup-git
```

Make Your Practice Repo (2)

```
$me = gh api user -q .login
$repo = "$me/orders-dotnet"
git clone https://github.com/coding-agent-harness-class/orders-dotnet $HOME\orders-dotnet
Set-Location $HOME\orders-dotnet
gh repo create $repo --private
git remote remove origin
git remote add origin "https://github.com/$repo.git"
git push -u origin main
gh repo set-default $repo
```

- Java: `orders-java` everywhere it says `orders-dotnet`
- Post your repo's link in the chat when it opens with `gh repo view --web`

One Thing to Expect

- Copilot won't give you exactly what it gave the person next to you
- Different runs and different models take different paths
- The skill is reading what the agent did, judging it, and steering it
- Activity steps describe what you should **see**, not exact output

Let's Begin

- Courseware: `README.md` at the root of the repository
- Activities: `courseware/activities/`
- Reference reading: `courseware/docs/`
- Questions any time, in the chat or out loud

The Copilot Harness and Chat in VS Code

GitHub Copilot as a Coding-Agent Harness

Xebia

The Copilot Harness and Chat in VS Code

Agenda

- Model versus harness
- One harness, four surfaces
- The capability ladder: chat → agent mode → CLI → cloud agent
- Chat that knows the project
- Choosing a model
- Live demo, then your turn

Model versus Harness

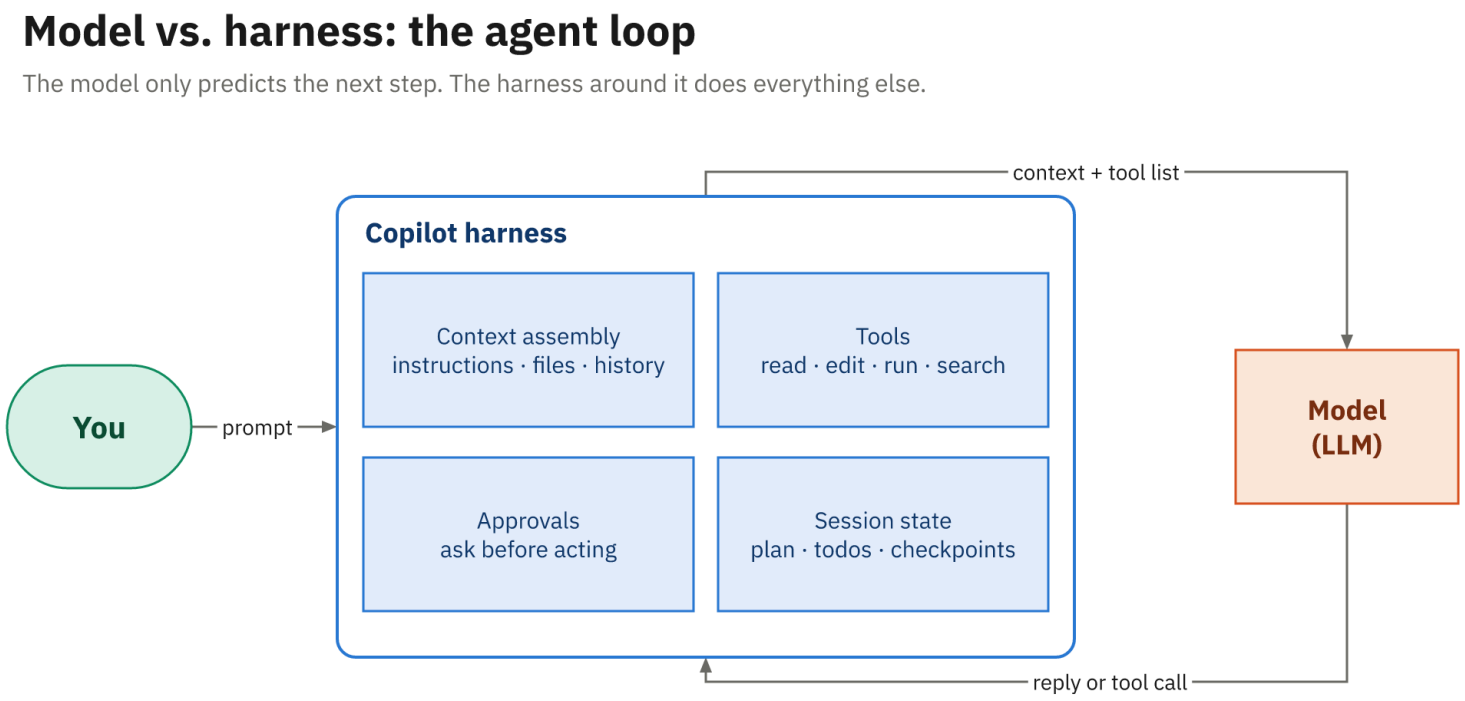
The Model Only Predicts

- Input: instructions, code, history, and a list of tools
- Output: an answer, or a request to call a tool
- It can't open a file or run `dotnet test`
- It remembers nothing between requests
- If a file isn't in the request, it hasn't seen it

The Harness Does the Rest

Job	What the harness does
Context assembly	Builds each request: instructions, files, history
Tools	Reads, searches, edits, runs commands, calls MCP
Agent loop	Feeds each result back until done or a limit
Approvals	Decides what runs now and what asks you first
Session state	Keeps history, pending edits, and checkpoints

The Agent Loop

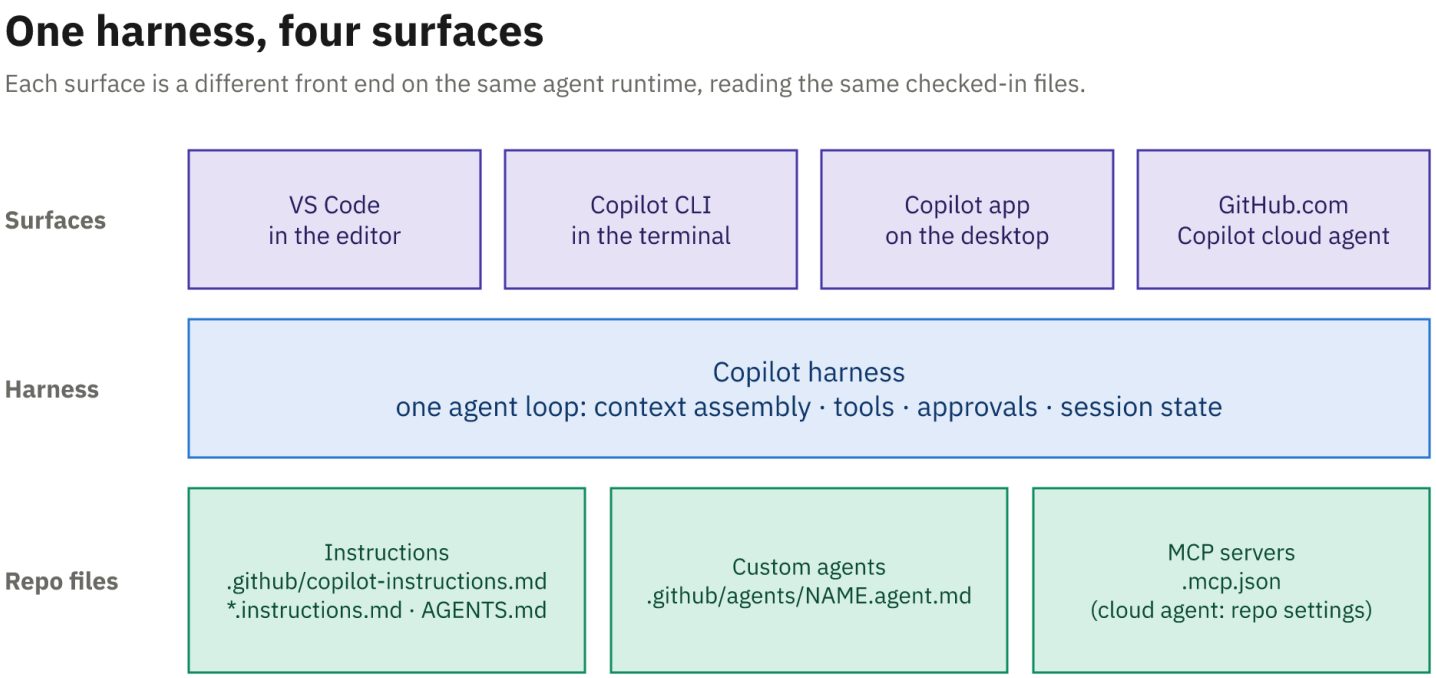


Swap the model and the loop stays the same. The harness decides what the model sees and what it's allowed to do.

One Request, Step by Step

- Harness builds the request: instructions, prompt, tool list
- Model: "search the workspace for `LateFeeCalculator`"
- Harness runs it; model asks to read the file and its tests
- Model proposes a test file; harness marks it pending
- Model asks to run `dotnet test`; harness asks you first
- Output goes back; model fixes an error or reports done

One Harness, Four Surfaces



Learn the harness once. The customizations you check in follow you to every surface.

Same Runtime Everywhere

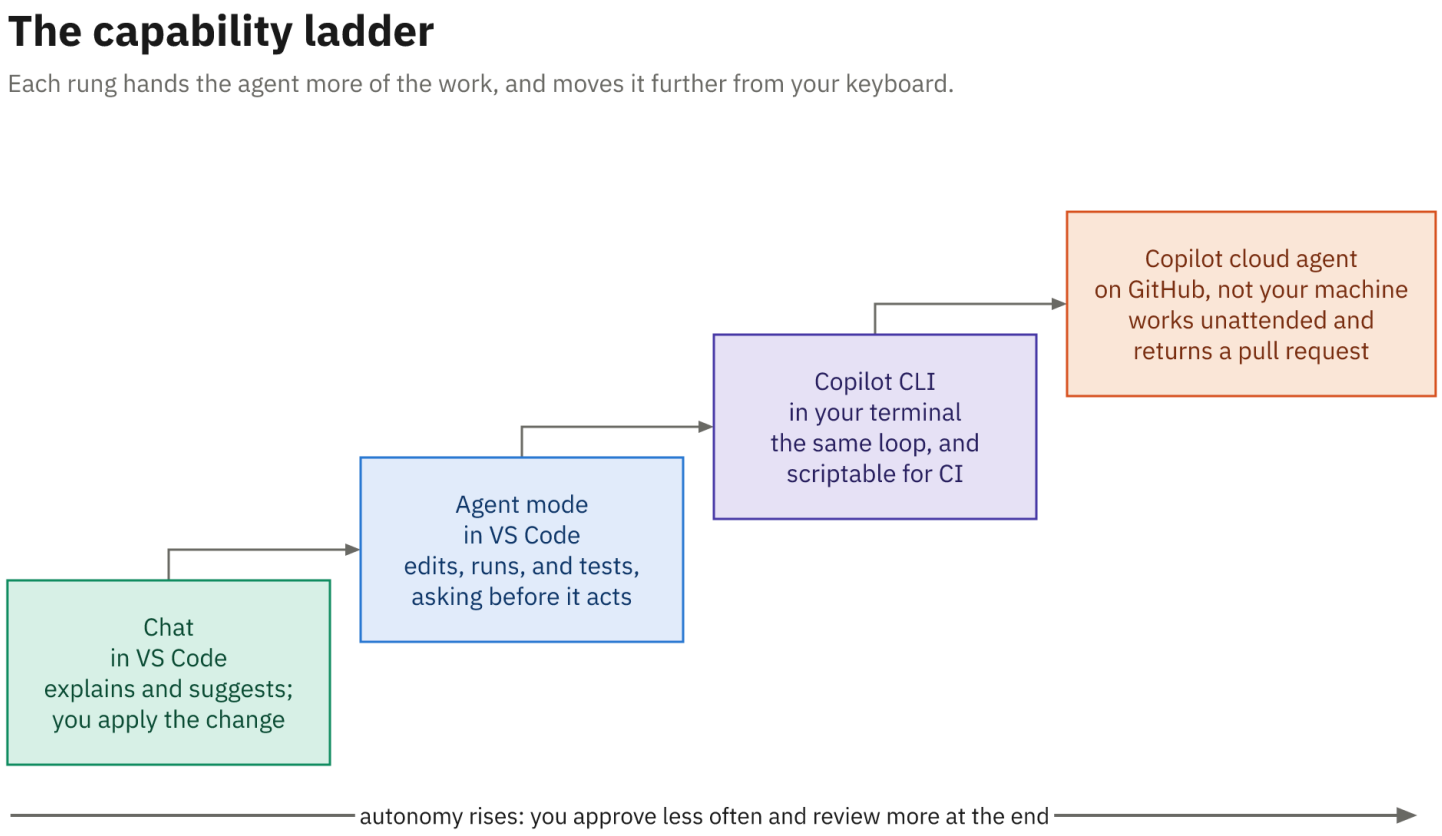
- One agent runtime, the engine the Copilot CLI hosts
- Copilot SDK (GA June 2026): drive that runtime from code
- VS Code's Copilot harness runs on the SDK and the Agent Host
- Cloud sessions share the CLI's tools: Bash, `gh`, code search
- Copilot app: desktop; each local session in its own worktree
- SDKs for .NET and Java, plus Node, Python, Go, and Rust
- Shared files: instructions, `.github/agents`, `.mcp.json`

Harnesses in VS Code

- Session Target picks the harness, before the first prompt
- Copilot: the default since 1.143 (SDK + Agent Host)
- Local: the original built-in agent, tied to the window
- Claude and Codex: Anthropic's and OpenAI's harnesses
- Cloud: a target, not a harness; runs in GitHub's cloud
- Prompt files, `/init`, and handoffs are Local-only
- This course uses the Copilot harness

The Capability Ladder

Four Rungs


The capability ladder
Each rung hands the agent more of the work, and moves it further from your keyboard.
Chat
in VS Code
explains and suggests;
you apply the change
Agent mode
in VS Code
edits, runs, and tests,
asking before it acts
Copilot CLI
in your terminal
the same loop, and
scriptable for CI
Copilot cloud agent
on GitHub, not your machine
works unattended and
returns a pull request
autonomy rises: you approve less often and review more at the end

What Changes at Each Rung

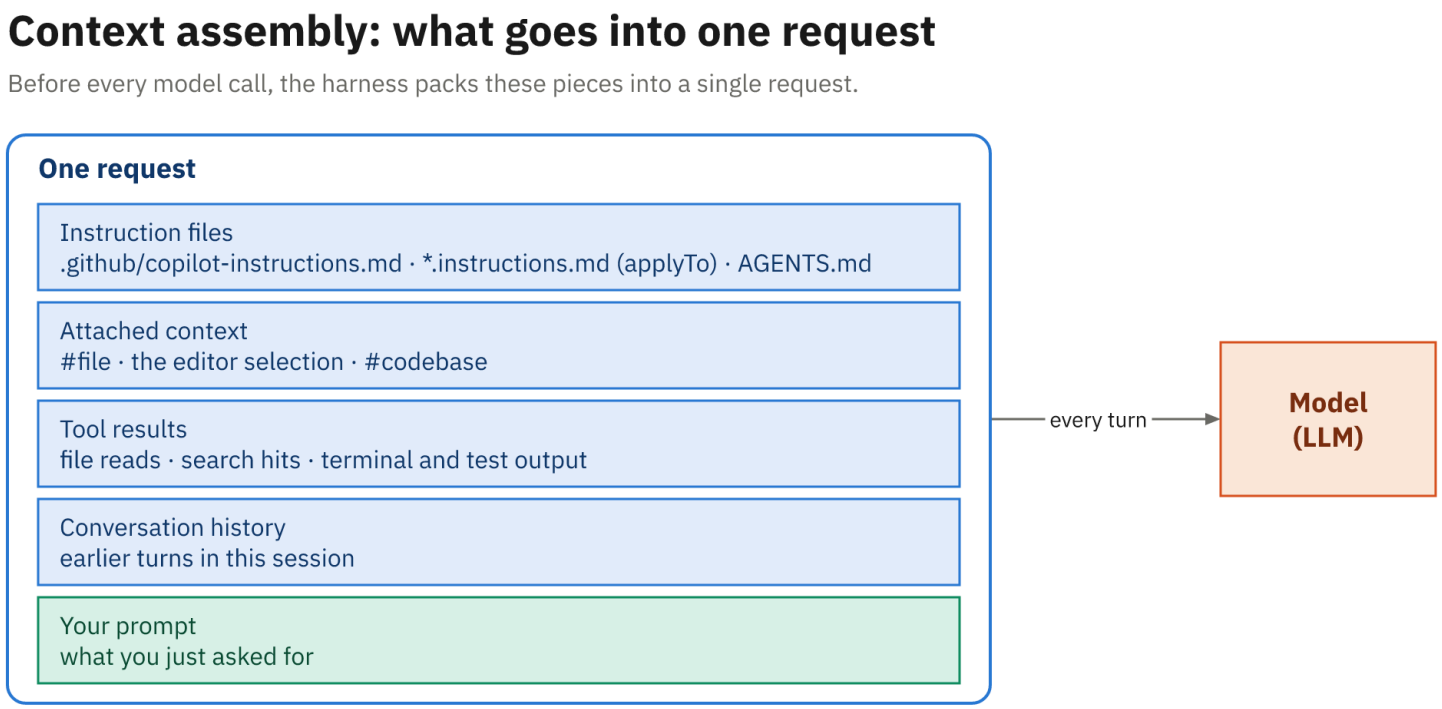
Rung	Who drives	Runs on	Can touch	You review
Chat	You	Your editor	Attached files, read-only	The answer
Agent mode	Agent + approvals	Your machine	Files, terminal, tests	Keep / Undo
Copilot CLI	Agent + approvals	Your terminal	Folder, shell, GitHub	Prompts, `/diff`
Cloud agent	Agent, async	GitHub's cloud	Its own branch	A draft PR

Picking a Rung

- A question about unfamiliar code → chat
- A small, well-scoped change you'll watch → agent mode
- Terminal work, a script, or a CI job → Copilot CLI
- A well-written issue to review later → cloud agent
- Each step up trades control for autonomy
- Sessions move: `/delegate` out, Continue in Copilot CLI back

Chat That Knows the Project

What Goes Into One Request



If something isn't in the request, the model doesn't know it. Instruction files make sure the basics always are.

Adding Context

Reference	Attaches
`#<file>`, `#<folder>`, `#<symbol>`	A file, a folder, a class, or a method
`#selection`	The highlighted code
`#codebase`	A search across the whole workspace
`#terminalSelection`, `#fetch`	Terminal text, or a web page

- Or use Add Context, or drag files, images, errors, issues in
- `@workspace` is deprecated: use `#codebase` instead

Inline Chat and Slash Commands

- `Ctrl+I`: inline chat in the editor or the terminal
- `Ctrl+Alt+I`: the Chat view · `Ctrl+N`: a new chat
- `/explain`: walk through the selection or file
- `/fix`: propose a fix for a problem
- `/tests`: generate tests (name xUnit or JUnit)
- `/doc`: doc comments, in editor inline chat
- Context menu: Generate Code > Generate Tests

Generate, Then Run

```
/explain #LateFeeCalculator.cs Who pays what, and when?
/tests Use xUnit v3. One [Theory] per rule.
/tests Use JUnit 6 and AssertJ with @ParameterizedTest.
```

```
dotnet test          # .NET
.\mvnw.cmd test      # Java on Windows (./mvnw test elsewhere)
```

- A generated test describes what the code does today
- Check expected values against the policy, not the code

Choosing a Model

The Model Picker

- VS Code: the Language model control in the Chat view
- CLI: `/model`, or `--model` at startup
- GitHub.com chat and the cloud agent: a model dropdown
- Vendors: OpenAI, Anthropic, Google, Microsoft, Moonshot, xAI
- GitHub sets the menu, and it changes often
- Some surfaces add a reasoning-effort setting

Auto

- Picks a model per task: system health + task complexity
- Profiles: Efficiency, Balance, Intelligence
- 10% off model costs on paid plans
- Billed at the rate of the model Auto chose
- Copilot Fusion: can combine several models, each billed
- A sensible default for this course

AI Credits

- Billing since June 1, 2026: tokens at each model's API rate
- Completions and Next Edit Suggestions use no credits
- Pro $10 · Pro+ $39 · Business $19/user · Enterprise $39/user
- 1 credit = 1¢; each plan includes at least its price
- No fallback to a cheaper model when credits run out
- Agent sessions make many model calls, so they cost more
- "Premium requests" and multipliers are legacy billing

Fast or Strong?

Task	Reach for
Explain, rename, write a doc comment	A fast model, or Auto
Tests for clearly stated rules	Auto
A multi-file change, a subtle bug, a plan	A stronger model
Unfamiliar business rules	A stronger model
The agent loops or misreads the code	Step up and retry

- Same wrong answer from a strong model? Fix the context.

Bring Your Own Key

- VS Code: Chat: Manage Language Models
- Azure, Anthropic, Gemini, OpenAI, or a custom endpoint
- Chat and utility tasks only; completions still use GitHub
- Copilot harness: turn on an experimental setting first
- Business and Enterprise: an admin enables the BYOK policy
- CLI: `COPILOT_PROVIDER_*` (`copilot help providers`)

Live demo

- A copy of `samples/library-dotnet`; tour the chat controls
- Ask (read-only): `#codebase` how do loan limits and fees work?
- `/explain` on `LateFeeCalculator`: the LIB-12 rules
- `/tests` with xUnit `[Theory]`, then `dotnet test`
- Check the expected values against the policy
- One question on a small model, a large one, then Auto
- References, then a debug view: what the model was sent

Your turn

- Activity 01: Chat with Workspace Context (25 minutes)
- The orders sample, in .NET or Java
- Use workspace context to explain `DiscountPolicy`
- Generate a unit test for one of its rules
- Run it with `dotnet test` or `.\mvnw.cmd test`

Key Takeaways

- The model predicts; the harness does the work
- One harness: VS Code, the CLI, the Copilot app, GitHub.com
- Climb the ladder as tasks grow and get better specified
- Good answers need good context: `#file`, `#selection`, `#codebase`
- `/explain`, `/fix`, `/tests`, and `/doc` for everyday jobs
- Start on Auto; step up to a stronger model for hard problems
- Billing is AI Credits, metered by the token

Agent Mode in VS Code with Instructions, Custom Agents, and MCP

GitHub Copilot as a Coding-Agent Harness

Xebia

Agent Mode, Instructions, Agents, MCP

Agenda

- Agent mode: planning, tools, approvals, and the session log
- Repository instructions: three kinds of file
- Custom agents in `.github/agents`
- MCP servers: GitHub and your own systems
- Live demo, then your turn

Agent Mode

Starting an Agent Session

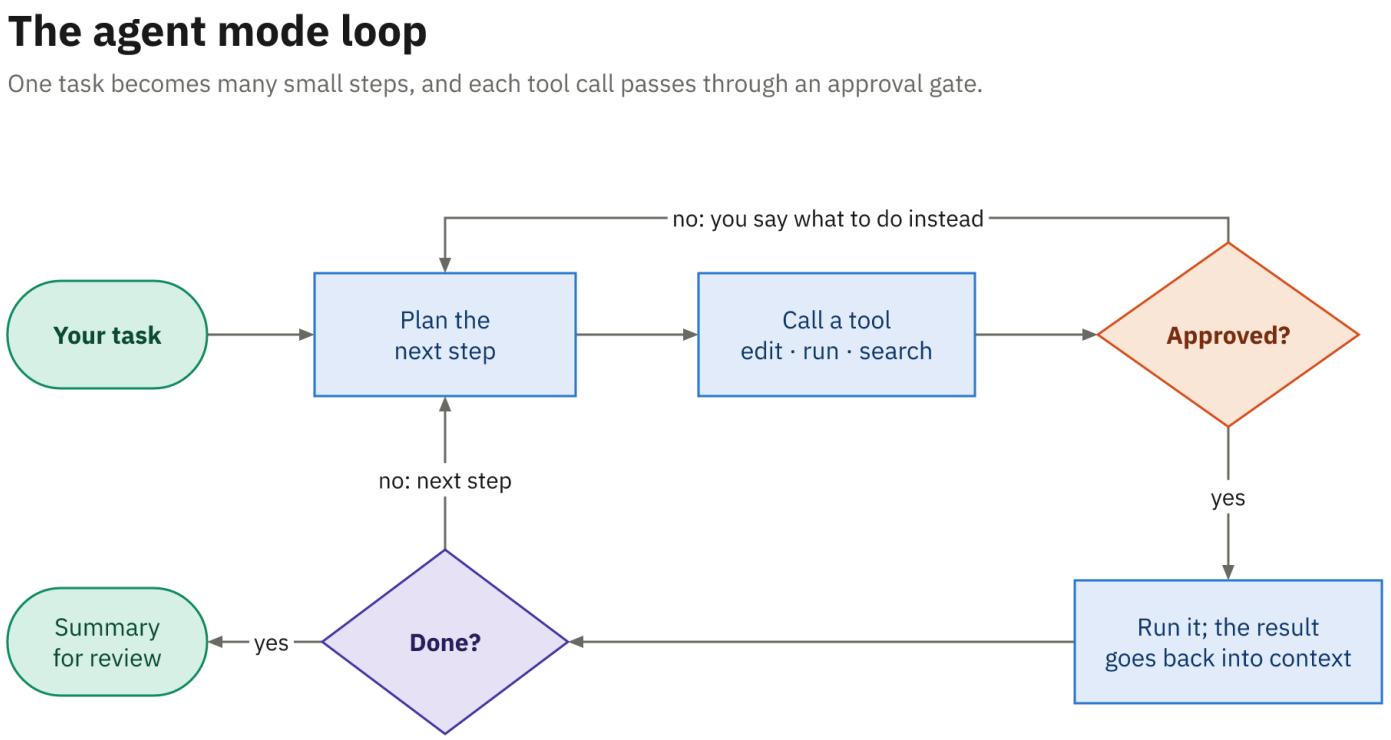
- Chat view (`Ctrl+Alt+I`), then set the session controls
- Session Target: Copilot, the default harness
- Agent: Agent (Ask is read-only; Plan, or `/plan`, writes a plan first)
- Language model: Auto, or your pick
- Permissions: Manual permissions to start
- Then give it a small, specific, checkable task

A Well-Scoped Task

```
Add a rule to LoanPolicy: a member who owes more than 10.00
in fines can't borrow. Report it as a new PolicyCodes
violation, add unit tests for both sides of the limit,
and run the tests when you're done.
```

- Names the class, the rule, the limit, and what "done" means
- Small enough to review in one sitting

The Agent Mode Loop



The approval gate is where you stay in charge: let the step run, or stop it and redirect the agent.

Tools the Agent Calls

Group	Examples (Local harness names)
Read and search	`#read/readFile`, `#search/codebase`, `#search/usages`
Edit	`#edit/editFiles`, `#edit/createFile`
Run	`#execute/runInTerminal`, `#execute/testFailure`
Track and delegate	`#todos`, `#agent/runSubagent`
Web and GitHub	`#web/fetch`, `#githubRepo`

- No run-tests tool: the agent runs `dotnet test` in the terminal

Permission Levels

Level	What it does
Manual permissions (default)	Follows your tool, URL, and terminal settings
Assisted permissions	An LLM judge assesses each tool call
Allow all	Runs every tool call without asking

- Approve once, or for the session, workspace, or all future uses
- Worktree sessions always run as Allow all
- Stay on Manual permissions in this course

Approval Settings

Setting	Default
`chat.tools.terminal.autoApprove`	`rm`, `curl` set to `false`; read-only commands run
`chat.tools.edits.autoApprove`	`{}` (glob patterns)
`chat.tools.global.autoApprove`	`false`, and leave it that way
`chat.agent.maxRequests`	`25` requests

- Chat: Manage Tool Approval · Chat: Reset Tool Confirmations
- Terminal sandbox: `chat.agent.sandbox.enabled`, off by default

Keep, Undo, Checkpoints

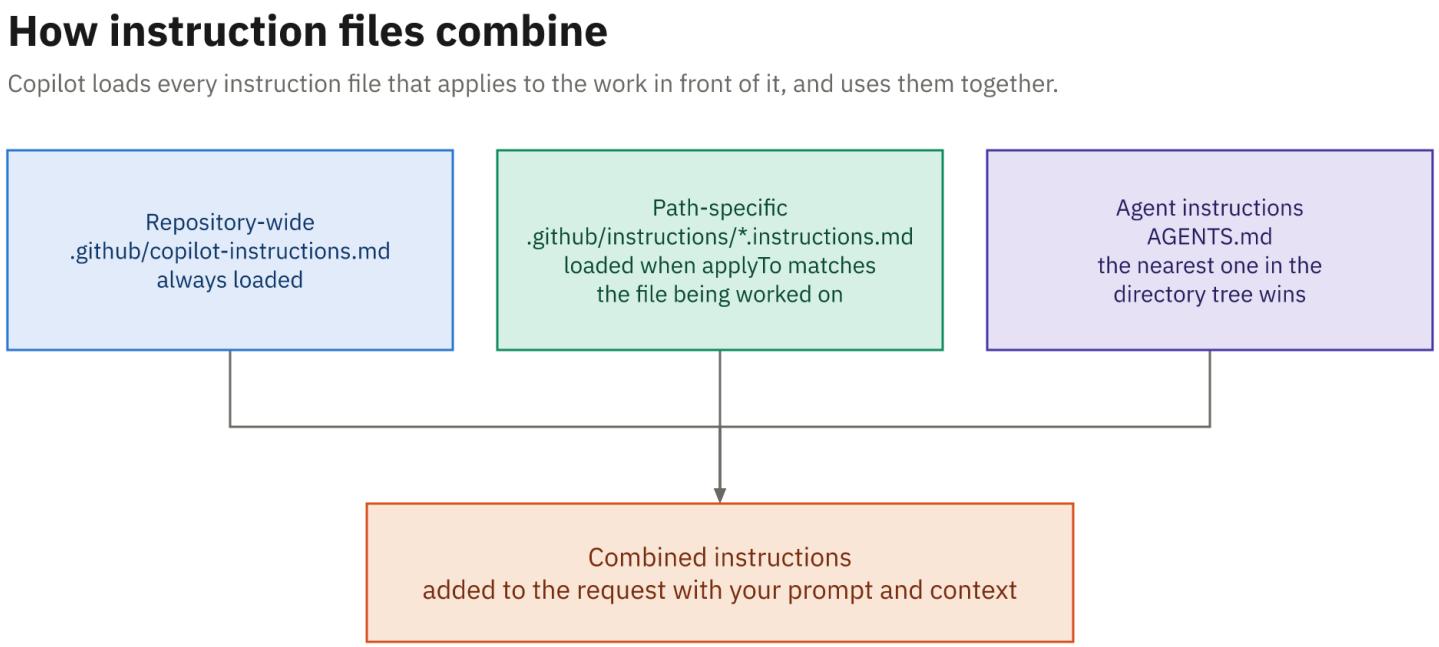
- Edits are on disk, but pending until you decide
- Keep: retains the changes and clears the pending state
- Undo: reverts them without rewinding the conversation
- Restore Checkpoint: hover an earlier request
- Checkpoints don't undo terminal commands or network calls
- Commit first, then review the diff like a colleague's PR

Reading the Session Log

- Chat Debug view: system prompt, context, response, tool results
- Open it from the overflow menu, or with `/debug`
- Check: did the instructions load? Which files did it read?
- Agent Debug Logs (Preview): tool calls, tokens, errors, flow
- Capture first: `chat.agentHost.agentDebugLog.enabled`
- Or ask: `/troubleshoot why didn't you run the tests?`

Repository Instructions

Three Kinds of Instruction File



Put rules that always apply in copilot-instructions.md, and rules for one part of the code in a path-specific file.

Which File Loads When

File	Loads
`.github/copilot-instructions.md`	On every request in the repository
`.github/instructions/*.instructions.md`	When the work matches its `applyTo`
`AGENTS.md` (one or more)	For its folder tree; the nearest wins

- VS Code and the CLI combine them all, with no precedence
- So don't let two files disagree

Repo-Wide Instructions

```
<!-- .github/copilot-instructions.md (excerpt) -->
## Loan rules
- Every rule lives in `LoanPolicy.CanBorrow` and reports a
  `PolicyViolation`. Never throw.
- Every new rule gets a new `PolicyCodes` constant (`LIB003`, …).
## Tests
- xUnit v3: test both sides of every limit in one `[Theory]`.
## Every change
- Add a line to `CHANGELOG.md`. Run `dotnet test` before you finish.
```

- Java: JUnit 6 `@ParameterizedTest`, `BigDecimal`, `.\mvnw.cmd test`

Path-Specific Instructions

```
# front matter of .github/instructions/tests.instructions.md
applyTo: "tests/**"                  # Java: "src/test/**"
```

- Body: use the in-memory stores, not mocks
- Body: control "today" with the `FixedClock` helper
- Applies when the work touches a matching file
- Repo-wide and path-specific both apply when both match
- Optional `excludeAgent`: `"code-review"` or `"cloud-agent"`

AGENTS.md

- One or more files; the nearest in the tree takes precedence
- Or a single `CLAUDE.md` or `GEMINI.md` at the repo root
- Portable: OpenCode and Kiro read `AGENTS.md` too
- Copilot Chat on GitHub.com reads repo-wide, not agent files

Who Reads What

Surface	Repo-wide	Path-specific	Agent files
VS Code chat and agent mode	Yes	Yes	`AGENTS.md` only
Copilot CLI	Yes	Yes	Yes
Cloud agent	Yes	Yes	Yes
Code review on GitHub.com	Yes	Yes	Yes, plus `REVIEW.md`
Chat on GitHub.com	Yes	No	No

- Check: References (VS Code), `copilot instruction list`

Custom Agents

A Custom Agent File

```
# .github/agents/policy-guard.agent.md (front matter, excerpt)
name: policy-guard
description: Read-only reviewer for the lending and fee policies.
tools: ["read", "search"]
```

```
You review changes to the Library lending service against its
business policies. You can read and search, but not edit or run.
```

- No `edit` or `execute` tools, so it can't change anything

Front Matter and Tool Aliases

Field	Notes
`description`	Required
`name`, `model`, `target`	Optional; `target` is `vscode` or `github-copilot`
`tools`	`read`, `edit`, `search`, `execute`, `agent`, `web`
`disable-model-invocation`	Plus `user-invocable`: who may invoke it

- MCP tools by server, such as `github/*`; `infer` is retired
- Prompt body: up to 30,000 characters

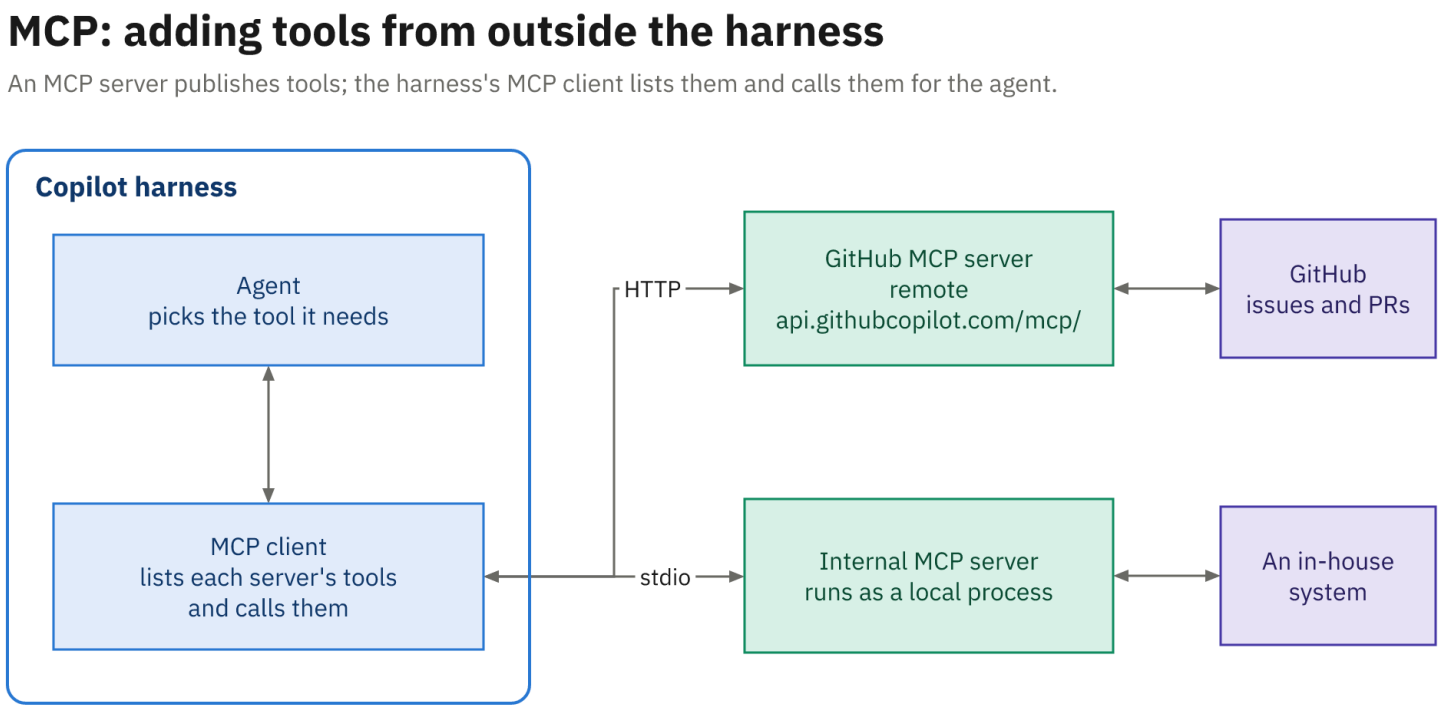
Picking an Agent Everywhere

Surface	How
VS Code	Session Target, then the Agent dropdown
Copilot CLI	`/agent policy-guard`, or `--agent=policy-guard`
GitHub.com	Agent dropdown: agents tab or issue assignment

- Org-wide: `agents/` in the org's `.github` or `.github-private`
- Name clash across levels: the closest level wins

MCP Servers

MCP in One Picture



The harness learns which servers exist from .mcp.json, and their tools appear beside the built-in ones.

The GitHub MCP Server

- Remote endpoint: `https://api.githubcopilot.com/mcp/`
- Default toolsets: context, repos, issues, pull requests, users
- Read-only: `…/mcp/readonly`; one toolset: `…/mcp/x/issues`
- Built in to the CLI; on by default for the cloud agent
- Open source (MIT); a local Docker image exists too

The Portable MCP File

```
{
  "mcpServers": {
    "github": {
      "type": "http",
      "url": "https://api.githubcopilot.com/mcp/"
    }
  }
}
```

- `.mcp.json` at the workspace root: VS Code and the CLI
- `.vscode/mcp.json` uses `servers`; the agent doesn't read it

An Internal Server (Config Only)

```
"board-policies": {
  "type": "stdio",
  "command": "dotnet",
  "args": ["run", "--project", "tools/BoardPolicies.Mcp"],
  "env": { "POLICY_API_TOKEN": "${POLICY_API_TOKEN}" },
  "tools": ["get_policy", "search_policies"]
}
```

- Secrets come from the environment, never from the file
- `tools` allowlists exactly what the agent may call

Governance and the Cloud Agent

- Policy "MCP servers in Copilot" turns MCP on or off
- Enterprise allowlists live in `managed-settings.json`
- Enforced in the Copilot app, the CLI, and VS Code
- Cloud agent: Settings → Copilot → MCP servers
- Every cloud server needs a `type` and a `tools` allowlist
- Cloud secrets are named `COPILOT_MCP_…`
- GitHub and Playwright MCP servers are on by default

Live demo

- Agent mode on the library app: the fines rule (`LIB003`)
- Approve or reject each step; watch `.\mvnw.cmd test` run
- Review the diff, then Keep or Undo
- Add `.github/copilot-instructions.md` and `tests.instructions.md`
- Stash the attempt, rerun the same prompt, and compare
- Pick the `policy-guard` agent to review the change
- Add `.mcp.json` and approve an MCP call that files an issue

Your turn

- Activity 02: Agent Mode with Instructions (25 minutes)
- The orders sample, in .NET or Java
- Agent mode adds a new validation rule with unit tests
- Approve or reject each step, then review the diff
- Add the repository instructions file and run the task again
- Compare: what changed because of the instructions?
- Add a custom agent and the GitHub MCP server, then push

Key Takeaways

- Agent mode runs the loop on your machine, behind your approvals
- Start on Manual permissions; read the diff; Keep or Undo
- The Chat Debug view shows exactly what was sent
- Instructions: repo-wide always, `applyTo` by path, `AGENTS.md` by tree
- Custom agents: a role, a prompt, and a limited set of tools
- `.mcp.json` is portable; the cloud agent's MCP lives in settings

GitHub.com, the Cloud Agent and Code Review

GitHub Copilot as a Coding-Agent Harness

Xebia

Cloud Agent and Code Review

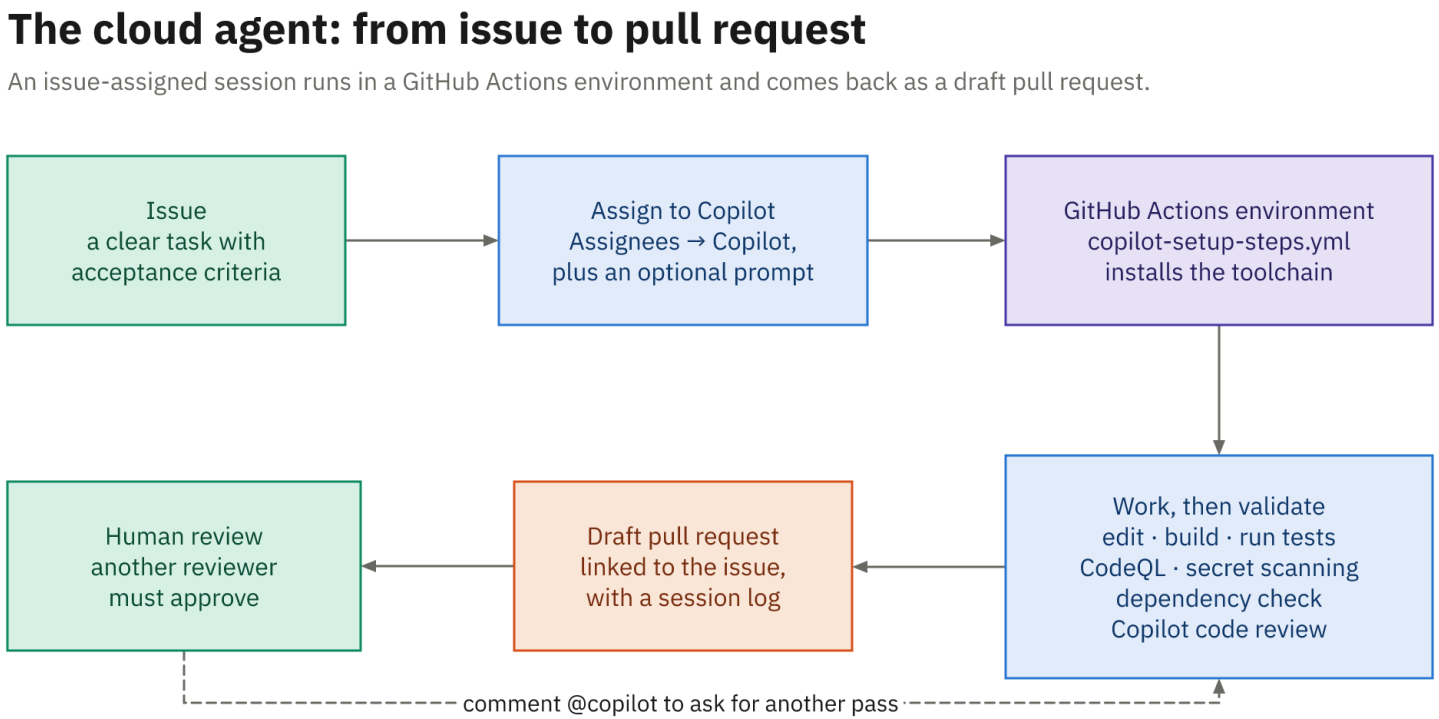
Agenda

- The Copilot cloud agent and where it runs
- Preparing the environment with `copilot-setup-steps.yml`
- Inside a run: validation and the session log
- Steering: issues, custom agents, Spaces, Memory
- Copilot code review and review instructions
- PR summaries and Chat on GitHub.com
- Other agents in the Agents panel
- Live demo, then Activity 03

What the Cloud Agent Is

- Formerly the Copilot coding agent; now **Copilot cloud agent**
- Part of **Copilot in the cloud** (2026-10-28); paid plans only
- Same harness: Bash, GitHub CLI, code search, your instructions
- Takes a task, works on its own, returns a pull request
- Needs write access; GitHub-hosted repositories only
- One branch and one pull request per task

Issue to Pull Request



Copilot pushes only to its own branch, and its pull request still needs a person's approval.

Assigning an Issue

- Issue → **Assignees** → **Copilot**
- The assignment options set:
  - an optional prompt
  - the repository and starting branch
  - a custom agent
  - the model and reasoning level
- Copilot reads the title, body, and existing comments
- Comments added after assignment are ignored

Other Ways to Start a Task

- The repository's **Agents** tab, or `github.com/copilot`
- `/task <description>` in Chat on GitHub.com
- **Fix with Copilot** on a failing Actions job
- `/delegate` in the Copilot CLI (Module 4)

```
gh agent-task create "Add GET /loans/overdue" --repo OWNER/REPO
gh agent-task list
```

- The Agents tab lists every session and links each to its PR

Where a Session Runs

	Actions session	Cloud sandbox
Typical start	Assigning an issue	Default for new sessions
Repositories	One	Several in one org
Setup steps	`copilot-setup-steps.yml`	Ignored
Time limit	59 minutes	No one-hour cap
Chat data kept	28 days	Life of the account

- **Environment** menu: Cloud (default), Actions, Environments

copilot-setup-steps.yml (1)

- Runs before the agent starts: SDKs, packages, caches
- One job, named exactly `copilot-setup-steps`
- Honored: `steps`, `permissions`, `runs-on`, `services`
- Also `snapshot` and `timeout-minutes` (59 max); rest ignored
- Must be on the default branch
- A failing step skips the rest; the agent carries on
- Actions sessions only; cloud sandboxes ignore it
- .NET sample: `setup-dotnet@v6`, then `dotnet restore`

copilot-setup-steps.yml (2)

```yaml
steps:
  - uses: actions/checkout@v7
  - uses: actions/setup-java@v6
    with:
      distribution: temurin
      java-version: "21"
      cache: maven
  - name: Warm the Maven cache
    run: sh ./mvnw -B -q -DskipTests verify
```

- The wrapper fetches Maven; `verify` fetches every dependency
- `sh ./mvnw`: a copy made on Windows loses the executable bit
- The runner is Ubuntu, so the agent builds with `sh ./mvnw`

The Firewall

- On by default: Settings → Copilot → **Internet access**
- **Recommended allowlist** is on by default
  - OS and language package registries, container registries, CAs
- **Custom allowlist** → **Add rule** for anything else
- Covers only processes the agent starts with its Bash tool
- Setup steps and MCP servers aren't covered
- Blocked requests show a warning on the pull request
- Doesn't work on Windows or self-hosted runners

The Pull Request It Returns

- A **draft** PR linked to the issue; Copilot requests your review
- Commits authored by Copilot, with you as co-author, signed
- Pushes only to its own branch (or the PR's branch)
- CI waits for **Approve and run workflows**; you can click it
- Your own approval of the PR **doesn't count**
- Cost: AI Credits for tokens plus Actions compute

Inside a Cloud Run

- Built-in validation on the agent's changes:
  - CodeQL and secret scanning
  - New dependencies checked against the GitHub Advisory Database
  - A Copilot code review second opinion
- No GHAS license needed; admins choose **Validation tools**
- Session log: **View session** on "Copilot started work"
- It streams live; read it for the plan, files, commands, blockers

Writing Issues It Can Act On

- The issue is the prompt: title, body, existing comments
- Plus the optional prompt you add when you assign it
- Say what "done" means, as checkable acceptance criteria
- Point at the files, and at existing code to reuse
- Say how to test, with the exact commands
- List what's out of scope
- Too vague: "Staff want to see overdue loans. Can we add this?"

A Good Issue for the Library App

```
# Add GET /loans/overdue
## Acceptance criteria
- 200 with a JSON array; empty when nothing is overdue
- Overdue = not returned and due date before today
- Each item: ids, dueOn, daysOverdue, feeIfReturnedToday
- Most overdue first; add a CHANGELOG line
## Where to look
- LoanService; reuse LateFeeCalculator (LIB-12)
## How to test
- Move the test clock past a due date; all tests pass
## Out of scope
- No changes to the lending rules; no new packages
```

Steering After Assignment

- Issue comments added later never reach the agent
- Comment `@copilot ...` on the pull request instead
  - Same branch, keeps context, write access only
- Works on any PR, including ones a person opened
- **Fix with Copilot** on a failing Actions job
- **Agent Merge**: **Watch and fix** failing CI or conflicts

Custom Agents, Spaces, Memory

- `.github/agents/NAME.agent.md` appears in the assign options
- `handoffs` and `argument-hint` are IDE-only
- Spaces: curated repos, issues, PRs, notes, and files
- Used from Chat on GitHub.com, and in IDEs via GitHub MCP
- Memory (GA): repository facts and personal preferences
- Expires after 90 days without reinforcement (orgs: 7–180)
- Preferences: on for Pro, Pro+, Max; Business needs the policy

Copilot Code Review

Requesting a Copilot Review

- PR → **Reviewers** → **Request** next to Copilot
- Usually finishes in under 30 seconds

```
gh pr edit 17 --add-reviewer '@copilot'   # quote it in PowerShell
```

- Automatic: on creation, **Review new pushes**, drafts
- Pushes don't re-trigger it by default, so re-request it
- Effort: Balanced by default; Lite, or Max (preview)

What a Review Gives You

- A **Comment** review by default; it doesn't count as approval
- Approvals: GA (2026-10-28), off until an admin turns them on
- Suggested changes commit in a couple of clicks
- **Fix with Copilot** hands a comment to the cloud agent
- **Add to batch**, then **Fix and commit**
- Or reply with `@copilot`, or decline with a reason
- Costs AI Credits plus Actions minutes

Review Instructions

- Reads `copilot-instructions.md` and `*.instructions.md`
- Plus `AGENTS.md`, `CLAUDE.md`, `GEMINI.md`, and `REVIEW.md`
- Read from the PR's **head** branch, not the base
- `excludeAgent: "code-review"` or `"cloud-agent"` targets a file
- Keep each file short: about 1,000 lines at most
- VS Code reviews may read only `copilot-instructions.md`

REVIEW.md for the Library App

```
# Review guidance: library lending service
## Money
- Fees are decimal (C#) or BigDecimal (Java); flag double, float
- Fee rules live only in LateFeeCalculator (LIB-12)
## Policy codes
- Never change LIB001 or LIB002; new rules get the next code
## Changelog
- Behavior or endpoint changes need a CHANGELOG.md line
```

- Concrete, checkable rules beat "write clean code"

More on GitHub.com

PR Summaries and Chat

- PR description toolbar → **Copilot actions** → **Summary**
- Start blank; regenerate after big pushes; not on Free
- Chat at `github.com/copilot`, available from any page
- Context: the repository, issue, or PR you're looking at
- A question can turn into agent work in the same session
- `/task <description>` starts a cloud-agent session
- Reads repo-wide, personal, and organization instructions

Other Agents in the Agents Panel

- Anthropic **Claude** and OpenAI **Codex** coding agents
- Public preview; Pro, Pro+, Max, Business, Enterprise
- Allowed by policy, per account, organization, or enterprise
- Start them on the Agents tab, from an issue, or `@AGENT_NAME`
- Same security validation; Actions minutes and AI Credits
- Run one issue through two agents and compare the PRs

Live demo

- Library app: the "list overdue loans" issue, assigned before class
- The **Agents** tab: find the session and open it
- The session log: plan, files read, test runs, checks
- A second run whose dependency check fires; **Validation tools**
- The draft PR: commits, co-author, linked issue
- **Approve and run workflows**, then watch CI
- Request a review: **Reviewers** → **Copilot**
- Make it automatic: a ruleset, or your personal settings
- **Copilot actions** → **Summary** on the PR description
- Chat on GitHub.com: ask how the late-fee rules work

Your turn

- **Activity 03: Issue to Pull Request, Reviewed by Copilot** (orders app)
- Your personal repo can't run the cloud agent, so agent mode does the work
- Before the break:
  - Create the issue from the provided markdown
  - Hand it to agent mode on a new branch
- After the break:
  - Finish, push the branch, and open the pull request
  - Request a **Copilot code review** and answer its comments
  - Compare with the cloud agent's pull request from the demo

Key Takeaways

- Assign an issue to Copilot and get back a draft pull request
- Issue-assigned sessions run on Actions, with a 59-minute cap
- `copilot-setup-steps.yml` gets the build ready first
- Validation and the session log keep unattended work honest
- The issue is the prompt: criteria, pointers, tests, scope
- Steer with `@copilot` on the PR, not on the issue
- Code review comments by default; `REVIEW.md` sets the bar
- You approve the workflow run; someone else approves the PR

The GitHub Copilot CLI in the Terminal

GitHub Copilot as a Coding-Agent Harness

Xebia

The GitHub Copilot CLI in the Terminal

Agenda

- What the CLI is, and how it differs from `gh`
- Installing, signing in, and trusting a folder
- Interactive sessions and programmatic runs
- Permissions, allow and deny rules, the local sandbox
- GitHub and MCP, built in
- Module 2's files, second opinions, and handoffs
- Live demo, then Activity 04

The Same Harness, in a Terminal

- One binary, `copilot`, generally available since February 2026
- Same runtime the Copilot SDK exposes and VS Code's harness uses
- Same loop: read, run, edit, test, and ask before risky steps
- Same repo files: instructions, custom agents, `.mcp.json`
- Sessions move between the CLI, VS Code, the app, and GitHub.com
- Runs as you, on your machine, with your toolchain

`copilot` Versus `gh`

Command	What it is	Use it for
`copilot`	The Copilot CLI	Agent work: read, edit, build, test
`gh`	The GitHub CLI	Exact GitHub operations
`gh agent-task`	A `gh` command	Start or follow cloud tasks
`gh pr edit --add-reviewer '@copilot'`	A `gh` command	Request Copilot code review
`gh copilot`	Retired extension	Nothing; stopped 2025-10-25

Install, Sign In, Trust

```
winget install Microsoft.PowerShell   # Windows needs PowerShell 6+
winget install GitHub.Copilot
copilot --version
copilot login                         # approve in your browser
```

- macOS: `brew install --cask copilot-cli`; npm needs Node 22+
- Script: `curl -fsSL https://gh.io/copilot-install | bash`
- Tokens: `COPILOT_GITHUB_TOKEN` > `GH_TOKEN` > `GITHUB_TOKEN`
- Trust loads the repo's MCP servers and hooks: launch at the root

A Session in the Terminal

Type	Example	Effect
`@file`	`@src/.../LateFeeCalculator.java`	Adds a file to context
`#number`	`#42`	Pulls in an issue or PR
`!command`	`!.\mvnw.cmd test`	Runs in your shell, not the agent
`&prompt`	`& Add doc comments to LoanPolicy`	Delegates to the cloud agent

- Shift+Tab cycles standard → plan → autopilot
- `--continue` or `--resume` brings a saved session back

Slash Commands You'll Use

Command	What it does
`/model`	Pick a model or `auto`; shows per-token cost
`/agent`	Choose a custom or built-in agent
`/instructions`, `/env`	See which files, servers, and agents loaded
`/plan`	Write an implementation plan before coding
`/diff`, `/review`	Inspect and review your changes
`/rewind`	Revert the last turn's file changes
`/usage`, `/context`	AI credits used; context window in use

Running a Prompt From a Script

```
$notes = copilot -p "Draft release notes for this branch versus main" `
    -s --allow-all-tools `
    --deny-tool='write' --deny-tool='shell(git push)'
```

- `-p` runs one prompt and exits; `-s` prints only the answer
- `--output-format json` emits JSONL for other tools
- Prompt mode needs `--allow-all-tools`; deny rules still win
- Also: `--model`, `--agent`, `--no-ask-user`, `--max-ai-credits`

In CI: An Illustration

```yaml
- name: Explain the failing tests
  if: failure()
  env:
    COPILOT_GITHUB_TOKEN: ${{ secrets.COPILOT_CLI_TOKEN }}
  run: |
    npm install -g @github/copilot
    copilot -p "Explain each failing test in target/surefire-reports" \
      -s --no-ask-user --allow-all-tools --disable-builtin-mcps \
      --deny-tool='write' --deny-tool='shell(git:*)'
```

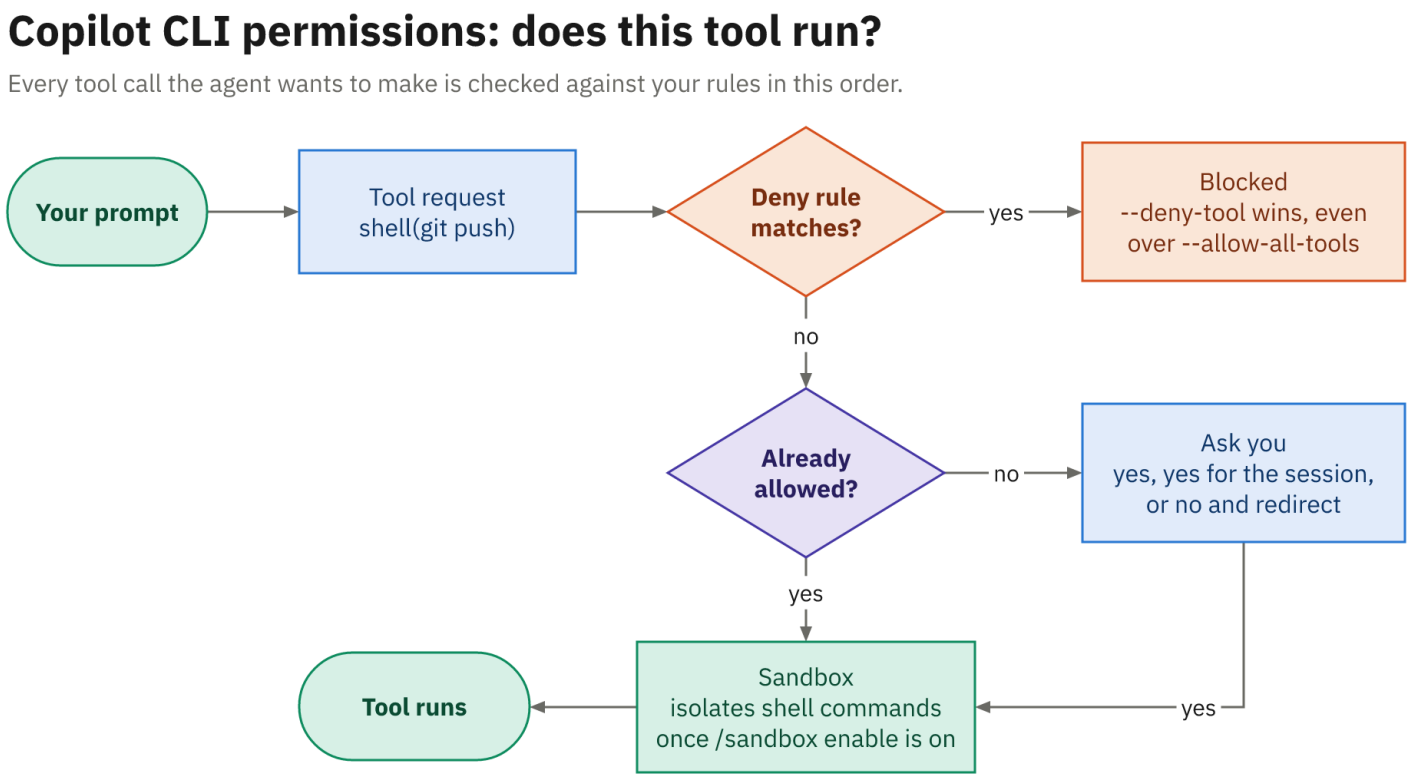
- Token: fine-grained PAT with Copilot Requests, as a secret
- Job `permissions: contents: read`; it explains, it never pushes

The Approval Prompt

```
1. Yes
2. Yes, and approve TOOL for the rest of the running session
3. No, and tell Copilot what to do differently (Esc)
```

- Option 3 declines and redirects the agent in one step
- `/permissions`: manual (default), assisted, or allow-all
- Assisted uses an LLM judge, which isn't a security boundary
- `/yolo` equals `/allow-all`; `/reset-allowed-tools` takes it back

How a Tool Request Is Decided



Deny rules always win. Allow rules, earlier approvals, and most reads skip the prompt; anything else asks you.

Allow and Deny Rules

```
copilot --allow-tool='shell(git:*)' --deny-tool='shell(git push)'
copilot --allow-tool='shell(dotnet:*)' --deny-tool='write(.env)'
copilot --allow-tool='inventory' --deny-tool='inventory(delete_item)'
```

- Kinds: `shell(...)`, `write(...)`, an MCP server name, `url(...)`
- Deny always wins, even over `--allow-all-tools`
- Paths: working directory by default; widen with `--add-dir`
- URLs: `--allow-url` and `--deny-url`, protocol-aware

Preview Before It Runs

- Plan mode (Shift+Tab or `/plan`): read the plan before any edit
- Approve one call at a time while the bug is still unfamiliar
- Decline with option 3 and say what to do instead
- `/diff` before you commit; `/rewind` reverts the last turn's edits
- A pushed branch isn't undone by `/rewind`: deny `git push` early

The Local Sandbox

- Off by default: `/sandbox enable`, or `--sandbox` for one session
- Shell commands run in an OS sandbox (MXC); file edits best-effort
- Windows 11 25H2 or 26H1 with recent updates; not Windows Server
- Starting policy: cwd, PATH, temp, and user profile; rest blocked
- `git` and `gh` authenticate through a credential-masking proxy
- `allowDevToolAccess` (on) grants NuGet and Maven caches per command
- Launch where `pom.xml` lives, or `~/.m2` isn't granted
- Org policy can require it and forbid per-command bypass

GitHub and MCP, Built In

- Built-in `github-mcp-server`: a read-only subset by default
- `#42` brings an issue or PR into the conversation
- `/pr create` opens the current branch's pull request
- Write tools: `--add-github-mcp-toolset pull_requests` (or `all`)
- Your servers: `copilot mcp add`, or a repo `.mcp.json`
- Not read: `.vscode/mcp.json`; project servers need a trusted folder

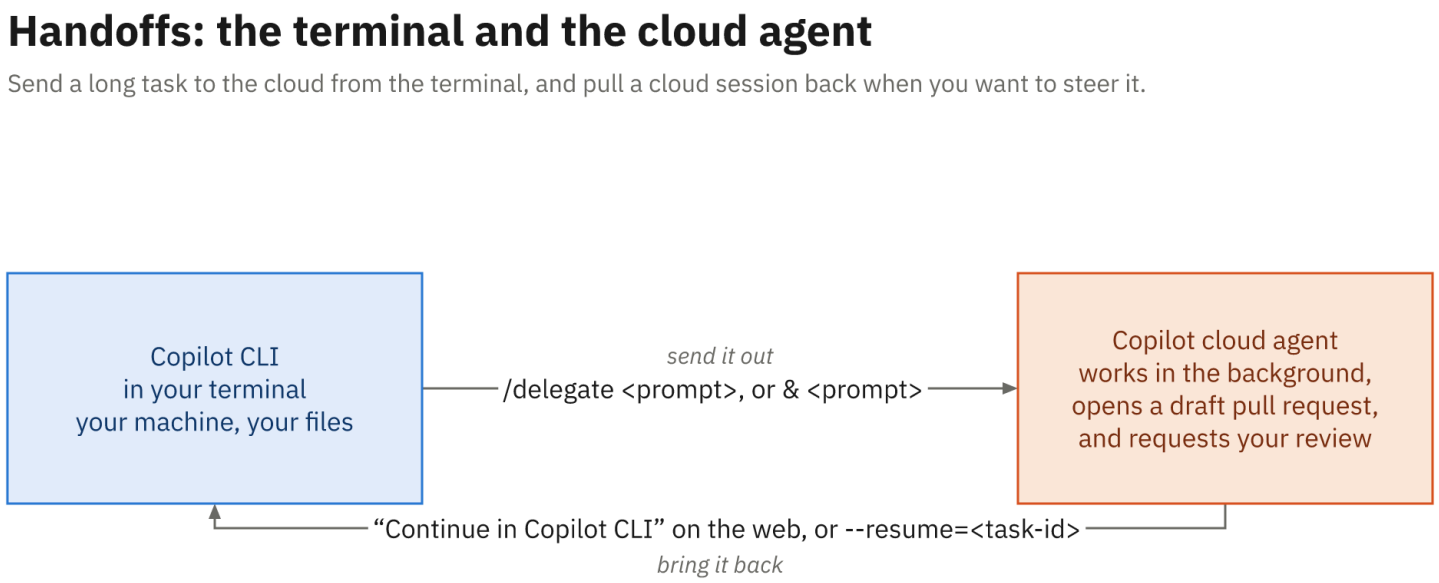
Same Files as Module 2

- `.github/copilot-instructions.md`: requests in the repo
- `.github/instructions/tests.instructions.md`: matching files
- `AGENTS.md`: root, working directory, and folders in between
- `.github/agents/test-writer.agent.md`: `/agent` or `--agent`
- `.mcp.json`: the same `mcpServers` file VS Code reads
- Check with `copilot instruction list`, `/instructions`, `/env`

Second Opinions and Handoffs

- `/review`: the built-in code review agent on your changes
- `/security-review` and `/rubber-duck` for other angles
- `/delegate` or `&`: send the session to the cloud agent
- It checkpoints your changes on a new branch, opens a draft PR
- `--base BRANCH` picks the pull request's target branch
- Back again: Continue in Copilot CLI, `/resume`, `--resume`
- `copilot --cloud`: the whole session in a cloud sandbox
- `/remote`: steer a running local session from web or Mobile

Terminal and Cloud, Both Ways



/delegate offers to commit your unstaged work to a new branch first, so the cloud agent starts where you stopped.

Live demo

- Library app (Java): `copilot --sandbox` in the repo
- `/instructions` and `/sandbox`: Module 2 files loaded, sandbox on
- `#NUMBER`: an issue that describes a failing test
- Plan mode to diagnose it before any edit
- Fix under step-by-step approval, declining one tool call
- An approved command the sandbox still blocks
- Run `.\mvnw.cmd test`, then open a PR from the terminal
- `/review`, `/delegate` out, Continue in Copilot CLI back

Your turn

- Activity 04: Fix a Reported Bug from the Terminal (25 minutes)
- Start `copilot` in your repo; read the issue with `#NUMBER`
- Diagnose the failing test in plan mode, then fix it under approval
- Run the suite: `dotnet test` or `.\mvnw.cmd test`
- Open a pull request without leaving the terminal
- Hand a follow-up to the cloud agent with `/delegate`
- Finished early? `/review`, `/security-review`, `copilot --cloud`

Key Takeaways

- The CLI is the same Copilot harness, running in a terminal
- `copilot` is the agent; `gh` still starts tasks and requests reviews
- `-p` scripts it: `--allow-all-tools` plus deny rules that always win
- Approvals decide what runs; the sandbox limits what it can touch
- Module 2's instructions, agents, and `.mcp.json` work unchanged
- `/review` for a second opinion; `/delegate` and `/resume` to hand off

Copilot, OpenCode, and Kiro Side by Side

GitHub Copilot as a Coding-Agent Harness

Xebia

Copilot, OpenCode, and Kiro Side by Side

Agenda

- Three harnesses at a glance
- Models and where they run
- Context: instruction files and MCP
- Approvals, sandboxing, and the path to a pull request
- What's open source
- Where Copilot stands out, and where it's constrained

Three Harnesses at a Glance

	Copilot	OpenCode	Kiro
Maker	GitHub	Anomaly	AWS
Surfaces	VS Code, CLI, app, GitHub.com	Terminal UI, desktop, web	IDE, CLI, Web, Mobile
Terminal command	`copilot`	`opencode` (v1 and v2)	`kiro-cli`
Windows install	WinGet `GitHub.Copilot`	Binary or npm	PowerShell script

- Same loop in all three: model, tools, approvals, session state

Models and Where They Run

	Copilot	OpenCode	Kiro
Model list	GitHub's: 6 vendors + Auto	Any provider	Kiro's, by credit multiplier
Your key, or local	BYOK in VS Code, CLI, app	Yes, incl. Ollama	Not documented
Hosting	Vendor clouds, GitHub's Azure	Your provider or machine	AWS, US and EU regions
Billing	AI Credits, token-metered	Your provider's bill	Monthly credits by tier

Context: Instructions and MCP

	Copilot	OpenCode 2	Kiro
Instructions	`copilot-instructions.md` and more	`AGENTS.md` only	`.kiro/steering/*.md`
`AGENTS.md`	Read; nearest wins	The only file	Always included
Custom agents	`.github/agents/`	`.opencode/agents/`	`.kiro/agents/`
MCP file	`.mcp.json`	`opencode.json`	`.kiro/settings/mcp.json`
MCP key	`mcpServers`	`mcp.servers`	`mcpServers`

- `AGENTS.md` is the one file all three read

Approvals, Sandbox, and the PR

	Copilot	OpenCode	Kiro
Approvals	Prompts, allow/deny, modes	allow / ask / deny rules	Autopilot or Supervised; rules
Local sandbox	Yes, off by default	None	Yes, off by default
Task to PR	Assign issue → draft PR	Local `git` and `gh`	`kiro` label or `/kiro`
Cloud runs	Actions or cloud sandbox	Action removed in v2.2.0	Kiro Web sandbox

What's Open Source

	Open source	Not open source
Copilot	VS Code + chat, SDK, MCP server	CLI, app, cloud agent, models
OpenCode	The whole agent (MIT)	Console, Go, Models
Kiro	Kiro Crew (Apache-2.0)	IDE, CLI, Web

Where Copilot Stands Out

- One harness from VS Code to the terminal to GitHub.com
- Issue to draft PR with CodeQL, secret and dependency checks
- Copilot code review as a built-in second opinion
- Six model vendors and Auto from one subscription
- Same instructions, agents, and MCP config across surfaces
- Org policy and MCP allowlists enforced in app, CLI, VS Code

Where Copilot Is Constrained

- Cloud agent: GitHub-hosted repositories only
- Actions sessions: one repo, 59-minute cap (issue assignment too)
- Cloud sandboxes span one org's repos, but skip setup steps
- One branch and one pull request per task
- Model menu set by GitHub; no BYOK in the cloud agent
- CLI, app, and agent services are proprietary
- Two meters: AI Credits plus Actions compute

Choosing for Your Team

- Copilot: code on GitHub; want issue to PR with checks built in
- OpenCode: need open source, any provider, or local models
- Kiro: spec-driven work, AWS hosting, or GitLab for cloud work
- OpenCode has no sandbox; Copilot's and Kiro's are off by default
- Keep conventions in `AGENTS.md` to stay portable

Key Takeaways

- All three run the same loop: model, tools, approvals
- They differ in who picks the model and where it runs
- `AGENTS.md` is the portable instruction file
- Copilot's edge is GitHub itself: issues, PRs, checks, review
- Its limits: GitHub-hosted repos, GitHub's menu, closed services